bc — Reference

bc is a text scientific calculator built with `lex` + `yacc` + `cc` (`bc.l` / `bc.y`). It evaluates expressions with full operator precedence, named variables, and a library of scientific functions. With no argument it is an interactive REPL; with an argument it evaluates that and prints the result. It is a *calculator*, not a general programming language — several of the tutorial activities below are therefore adapted or noted as not applicable.

```
bc "sqrt(2) * 10"     # evaluate one expression and print
bc                    # interactive REPL (q or quit to exit)
```

Quick Reference

```
2 + 3 * 4         ^ = power (right assoc)   \ = none; use /   % = remainder
x = 5             # assign;  x += 1  -= *= /= %= ^=
pi  e             # built-in constants
sqrt sin cos tan asin acos atan exp ln log log2 abs floor ceil round int sign rad deg
pow(x,y) atan2(y,x) hypot(a,b) max(a,b) min(a,b) mod(a,b) logn(base,x) root(n,x)
==  ≠  <  ≤  >  ≥   &&  ||  !          # comparisons/logic yield 1 or 0
# comment to end of line
```

Data types

One type: a floating-point **number** (`System.Double`). Integers print without a decimal point; everything is computed in double precision and shown with up to 10 significant digits.

Statements / Commands

bc has no statements in the programming sense — each line is an **expression** whose value is printed, optionally with an assignment (`x = expr`). Multiple expressions can be separated by `;`. There are no loops or conditionals in this subset (see *Subset boundaries*); comparisons are operators that yield `1` / `0`.

Functions

A rich set of **built-in** scientific functions (you cannot define your own in this subset): one-argument `sqrt sin cos tan asin acos atan exp ln log log2 abs floor ceil round int sign rad deg`; two-argument `pow atan2 hypot max min mod logn root`.

Input / Output

Input is the expression you type (REPL) or pass as an argument; output is the computed value, printed with `%.10g` formatting (so `2.0` shows as `2`, `sqrt(2)` as `1.414213562`).

Graphics

None. bc is numeric only — Activity 8 below is a small **numeric table** instead.

Notes

- Floating point throughout (not bc's traditional arbitrary precision).
- `^` binds tighter than unary minus, so `-2^2 = -4` (matching real bc).
- Comparisons and `&& || !` produce `1` / `0`.
- `q` or `quit` exits the REPL.

Subset boundaries

A scientific *calculator*, not the full bc language. Not included: user-defined `define` functions; `if` / `while` / `for` statements; arrays; `scale` /arbitrary precision; strings/ `print`. State is just named variables.

Tutorial

Every example was run with `bc`; the output shown is real.

1. Your first program

```
bc "2 + 3 * 4"
```

```
14
```

bc honours precedence — multiplication before addition.

2. Variables and data types

In the REPL (one expression per line):

```
r = 5
area = 3.14159 * r * r
area
```

```
5
78.53975
78.53975
```

Assigning prints the assigned value; the bare name `area` re-prints it. Every value is a double.

3. Flow control

This subset has no loops or `if` — but comparisons are operators that yield `1` / `0`, which is the calculator's form of a decision:

```
bc "3 < 5"
```

```
1
```

(`&&`, `||`, `!` combine these; there are no statement-level control structures — see *Subset boundaries*.)

4. Arrays

bc has no arrays in this subset (see *Subset boundaries*) — it is a calculator over scalar numbers and named variables. Where another language would index an array, bc computes a single value per expression.

5. Subroutines and functions

You can't `define` functions here, but the **built-in** scientific library is the point of a scientific calculator:

```
bc "sqrt(144)"
bc "pow(2, 8)"
bc "sin(pi / 2)"
bc "ln(e)"
```

```
12
256
1
1
```

`pi` and `e` are built-in constants; `sin` / `cos` / `tan` take radians (use `rad(deg)` to convert), and `ln` / `log` / `log2` are natural/base-10/base-2 logarithms.

6. Memory management

There is nothing to manage — bc keeps a table of **named variables**, each holding one double. Assigning a name creates or updates its entry; there is no allocation, no arrays, and no freeing.

```
bc "x = 10; x += 5; x ^= 2"
```

```
225
```

`x` becomes 10, then 15, then $15^2 = 225$ — each compound assignment updates the same variable.

7. Strings and a text layout

bc has no strings — it is numeric only. "Layout" is numeric formatting: values print with up to 10 significant digits, integers without a decimal point:

```
bc "sqrt(2)"
```

```
1.414213562
```

8. Drawing a picture

No graphics and no strings, so the nearest thing is a **numeric table** — evaluating a function at several points:

```
bc "sin(rad(0))"
bc "sin(rad(30))"
bc "sin(rad(90))"
```

```
0
0.5
1
```

`rad` converts degrees to radians; the column of results sketches the rising sine curve from 0° to 90°.

9. Where to go next

Use bc as a desk calculator — chain expressions with `;`, keep intermediate results in variables, and reach for the scientific functions and `pi` / `e`. The natural extensions (user-defined `define` functions, `if` / `while`, arrays — the rest of the historical bc language) are listed under *Subset boundaries*.